OpenCAA Book Studio — Documentazione tecnica

Architettura, componenti, flussi applicativi, dipendenze e criticità implementative

Versione documentale: 1.0 | Data: 1 novembre 2026

1. Scopo e perimetro

Il sistema è una web application client-side realizzata in un singolo documento HTML (index.html), con CSS e JavaScript incorporati. Il secondo file, caa.html, è una pagina descrittiva/di presentazione: non implementa l'editor, ma illustra obiettivi, funzionalità e flusso logico del prodotto.

La presente documentazione distingue le funzionalità effettivamente riscontrabili nel codice dell'editor dalle descrizioni promozionali presenti nella pagina portfolio. Non vengono attribuite al sistema funzioni non implementate.

2. Struttura dei file

File	Ruolo	Tecnologie
index.html	Applicazione editoriale OpenCAA Book Studio.	HTML5, CSS3, JavaScript vanilla, DOM API, Fetch API, Web Speech API.
caa.html	Landing/documentazione descrittiva per stakeholder e portfolio.	HTML5, Tailwind CSS via CDN, CSS custom, SVG inline.

3. Architettura logica

L'applicazione non presenta separazione in moduli, bundler, framework, API server proprietarie o persistenza. Il flusso principale è sincrono nella gestione dell'editor, con richieste asincrone per la creazione della tessera e il recupero del pittogramma.

Sottosistema	Responsabilità	Implementazione
Interfaccia	Toolbar, area di editing, sidebar pagine e tessere.	Markup HTML e CSS nel documento.
Stato	Elenco pagine, indice attivo, visibilità simboli.	Oggetto JavaScript bookState.
Gestione pagine	Creazione, cambio pagina, serializzazione DOM corrente.	renderSidebarPages, addNewPage, switchPage, saveCurrentPageDOM, loadPage.
Parsing input	Rilevamento parola al tasto spazio/Invio.	Listener keydown, Selection e Range API.

Analisi linguistica	Normalizzazione, regole contestuali, mappatura lemma e categorie.	analyzeContextWithNLP e dizionari statici; fallback it-compromise.
Generazione tessera	Creazione del nodo non editabile con classe cromatica.	createTokenNode.
Recupero simboli	Ricerca ARASAAC, costruzione URL immagine e cache in memoria.	fetchSymbolImage e Map symbolCache.
Accessibilità/presentazione	Font, dimensioni, colore, allineamento, visibilità, TTS.	Custom properties CSS e Web Speech API.
Output	Stampa del contenuto attivo.	window.print e @media print.

4. Dipendenze e servizi esterni

Dipendenza	Uso	Note operative
it-compromise	Analisi NLP italiana e tag grammaticali.	Caricata da https://unpkg.com/it-compromise; il codice verifica l'esistenza globale nlp.
ARASAAC API	Ricerca dei pittogrammi in italiano.	GET https://api.arasaac.org/api/pictograms/it/search/{query}.
ARASAAC static assets	Download dell'immagine del pittogramma.	URL costruito come https://static.arasaac.org/pictograms/{id}/{id}_300.png.
Google Fonts	Atkinson Hyperlegible, Comic Neue, Open Sans.	Caricati da fonts.googleapis.com.
OpenDyslexic CDN	Font OpenDyslexic.	Caricato tramite @font-face da jsDelivr.

La disponibilità dell'editor e la completezza delle sue funzioni dipendono quindi anche dalla rete e dalla disponibilità dei servizi terzi. Non è implementata una modalità offline completa.

5. Modello dello stato

bookState contiene pages, currentPageIndex e showSymbols. Ogni elemento pages è un oggetto con id e html. html contiene la serializzazione innerHTML della pagina attiva. symbolCache è una Map che associa la query normalizzata del pittogramma al relativo URL.

Lo stato è volatile: non vengono usati localStorage, IndexedDB, file system, cookie applicativi o endpoint proprietari. Il salvataggio della pagina avviene solo quando si cambia pagina o se ne aggiunge una, tramite saveCurrentPageDOM.

6. Flusso di elaborazione del testo

1. Il listener keydown è associato all'elemento #active-book-page, contenteditable=true.
2. Alla pressione di spazio o Invio viene letta la selezione corrente tramite window.getSelection() e Range.
3. Se il nodo attivo è un Text node, una regex individua l'ultima sequenza non vuota prima del cursore.
4. Il codice separa, quando il pattern corrisponde, la parola dai segni di punteggiatura ammessi.
5. Il contesto passato al motore è il textContent del parent node, non un'analisi completa e strutturata dell'intero documento.

6. createTokenNode richiama analyzeContextWithNLP, genera uno span .caa-token e avvia fetchSymbolImage.
7. La tessera viene inserita nel DOM e il cursore viene riposizionato dopo uno spazio non separabile o un newline.

7. Motore di disambiguazione e classificazione

analyzeContextWithNLP(word, fullSentenceContext) applica una sequenza ordinata di regole. La prima regola che restituisce un risultato interrompe la valutazione successiva.

Ordine	Regola	Effetto
1	LOGICAL_CONNECTIVES	Connettori come perché, ma, se, quindi, poi, e, o → misc.
2	COMPOUND_PREPOSITIONS	Preposizioni articolate (del, alla, nel, sul…) ricondotte alla preposizione base → misc.
3	PREPOSITION_LOCUTIONS	Dopo “a” o “in”, alcune forme (pesca, caccia, nuoto, ballo) sono ricondotte a un verbo → action.
4	PRONOUNS + VERB_LEMMA_MAP	Dopo un pronome, le forme verbali presenti nella mappa diventano lemma e categoria action.
5	ARTICLES	Dopo un articolo/determinante elencato, la parola è trattata come object e si tenta il singolare.
6	VERB_LEMMA_MAP	Mappatura diretta delle forme verbali note → action.
7	it-compromise	Lettura dei tag Verb, Noun, Honorific, Person, Pronoun, Adjective per stimare la categoria.
8–9	Fallback	Regola morfologica semplificata del singolare e liste Fitzgerald statiche.

La funzione getSingularLemma applica due trasformazioni: finale -i → -o e finale -e → -a, se la parola supera tre caratteri. Non è un lemmatizzatore morfologico generale. VERB_LEMMA_MAP copre un insieme finito di forme. La qualità del risultato varia quindi in base al lessico e alla costruzione della frase.

8. Recupero dei pittogrammi

8. La query ricevuta dal motore viene passata a fetchSymbolImage.
9. Se symbolCache contiene la query, il codice riutilizza l’URL già memorizzato.
10. In caso contrario viene inviata una richiesta fetch all’endpoint ARASAAC in lingua italiana.
11. Se la risposta è OK e contiene almeno un risultato, viene usato data[0]._id (primo elemento).
12. Il codice costruisce l’URL PNG statico e lo inserisce in un elemento img.
13. Se la richiesta fallisce o non restituisce risultati, viene visualizzato .symbol-placeholder.

Limite funzionale: non esiste un passaggio di selezione o conferma del pittogramma. Il primo risultato API viene accettato automaticamente.

9. Componenti dell’interfaccia e funzioni

Elemento/funzione	Descrizione tecnica
#active-book-page	Area contenteditable che ospita testo e span delle tessere.
#pages-list	Contenitore dinamico delle miniature pagina.
updateFontStyle()	Aggiorna CSS custom properties per famiglia, dimensione e colore testo.
setAlignment(align)	Aggiorna --doc-alignment.
updateSymbolSize()	Imposta --symbol-size, --token-width e --token-height per small/medium/large.
toggleSymbols()	Alterna la classe hide-symbols sulla pagina attiva e aggiorna bookState.showSymbols.
speakWord(text)	Crea SpeechSynthesisUtterance con lang='it-IT' e invoca speechSynthesis.speak.
readPage()	Concatena i contenuti .token-text della pagina attiva e li invia a speakWord.
window.print()	Apre il flusso di stampa del browser.

10. Stile e stampa

Il CSS definisce le classi .fitz-people, .fitz-action, .fitz-object, .fitz-descriptor, .fitz-social e .fitz-misc. Le variabili CSS in :root centralizzano palette, font, dimensioni e impostazioni del documento. La regola @media print nasconde toolbar e sidebar, rimuove ombre e padding e rende la pagina espandibile.

La stampa è demandata al browser: non è presente una libreria PDF, né una routine che generi un file PDF direttamente. La dicitura “PDF” si riferisce alla possibilità di selezionare una stampante PDF nella finestra di stampa.

11. Sicurezza, robustezza e qualità del codice

- createTokenNode inserisce la parola digitata in una stringa assegnata a innerHTML. Per un’applicazione distribuita è preferibile creare nodi DOM e usare textContent, evitando che input utente venga interpretato come markup.
- saveCurrentPageDOM e loadPage trasferiscono HTML tramite innerHTML. Se il contenuto può provenire da fonti non fidate, occorre introdurre sanitizzazione e un modello dati strutturato anziché HTML arbitrario.
- Le miniature sono div cliccabili e non sono implementate come controlli button con semantica nativa; occorre verificare navigazione da tastiera e annunci degli screen reader.
- Il codice non fornisce etichette ARIA complete, gestione del focus annunciata, stati di caricamento/errore accessibili o alternative manuali per il pittogramma.
- Le richieste esterne non hanno timeout, retry controllato o messaggi di errore visibili all’utente.
- Il codice non gestisce esplicitamente speechSynthesis.cancel(): letture successive possono sovrapporsi o accodarsi secondo il browser.
- Il rilevamento del contesto usa findIndex con includes(cleanWord), che può trovare corrispondenze parziali e non disambigua in modo robusto occorrenze ripetute.
- Il parser interviene solo quando il cursore si trova in un nodo di testo; incolla, modifica retroattiva, selezione multipla e input complessi non sono gestiti come un processo di conversione completo.

12. Accessibilità: stato e verifiche necessarie

Sono presenti alcune scelte orientate alla leggibilità (font selezionabili, dimensioni, contrasto configurabile, allineamento, simboli ridimensionabili e sintesi vocale). Queste caratteristiche non equivalgono a conformità WCAG 2.2 AA. Il codice va sottoposto a verifica manuale e automatizzata.

- Verificare uso completo da tastiera, ordine del focus, focus visibile e assenza di trappole.
- Associare label programmatiche ai controlli e usare button/select nativi con nomi accessibili.
- Rendere le tessere comprensibili agli screen reader e definire un comportamento coerente per il contenuto non editabile.
- Annunciare caricamento, errore e assenza di pittogramma tramite regioni live, senza sovraccaricare gli annunci.
- Verificare contrasto per tutte le combinazioni colore selezionabili, zoom 200–400%, reflow e orientamento.
- Testare con NVDA, VoiceOver e almeno un browser desktop e mobile supportato.
- Verificare che l'uso del colore non sia l'unico mezzo per comunicare la categoria.

13. Test funzionali consigliati

ID	Scenario	Risultato atteso / controllo
F-01	Digitare "Io vado a scuola "	Creazione di tessere; verificare classificazione e query ARASAAC.
F-02	Confrontare "vado a pesca" e "mangio la pesca"	Verificare la regola contestuale preposizionale/articolo e controllare manualmente i risultati.
F-03	Digitare un verbo non presente nella mappa	Verificare comportamento del fallback NLP e segnalare classificazioni errate.
F-04	Ricerca ARASAAC senza risultati / rete assente	Rimane il segnaposto senza bloccare l'editor.
F-05	Cambiare pagina e tornare alla precedente	Il contenuto HTML della pagina resta nella sessione.
F-06	Ricaricare la pagina	Verificare e documentare la perdita dello stato non persistito.
F-07	Cambiare font, dimensione, colore e allineamento	La presentazione della pagina attiva si aggiorna.
F-08	Nascondere e ripristinare i simboli	Le immagini scompaiono/riappaiono secondo lo stato UI.
F-09	Leggere una tessera e l'intera pagina	La sintesi vocale usa lingua it-IT, se supportata.
F-10	Stampare più pagine	Verificare che la stampa includa effettivamente tutte le pagine: il codice invoca print sul DOM corrente e non implementa una gestione esplicita di paginazione multi-pagina.

14. Debito tecnico e roadmap raccomandata

Priorità	Intervento	Motivazione
P0	Sostituire innerHTML per l'input utente con textContent/DOM API; introdurre sanitizzazione.	Ridurre rischio XSS e corruzione del markup.
P0	Aggiungere scelta manuale e correzione di pittogramma, lemma e categoria.	Necessario per controllo professionale del materiale CAA.
P0	Implementare salvataggio/esportazione/importazione del progetto.	Evitare perdita del lavoro e consentire riuso.
P1	Separare logica NLP, API, stato e UI in moduli JavaScript.	Manutenibilità e testabilità.
P1	Ampliare e testare regole linguistiche con corpus di frasi italiane e casi ambigui.	Ridurre errori di lemmatizzazione e classificazione.
P1	Migliorare semantica HTML, tastiera, screen reader e feedback accessibili.	Accessibilità effettiva e verificabile.
P1	Gestire errori/retry/timeout e stati di caricamento ARASAAC.	Robustezza operativa.
P2	Aggiungere gestione libri, riordino/eliminazione pagine, cronologia e template.	Migliorare il flusso editoriale.
P2	Predisporre test automatici unitari, integrazione e test browser.	Prevenire regressioni.

15. Avvio e manutenzione

Per la versione attuale è sufficiente distribuire index.html su un hosting statico HTTPS o aprirlo in un browser compatibile. Per un deployment stabile è preferibile servire il file via HTTP(S), fissare versioni delle dipendenze CDN, controllare le licenze, introdurre una Content Security Policy compatibile con le risorse necessarie e monitorare la disponibilità dei servizi esterni.

Non sono richiesti Node.js, un database o un processo di build nella versione consegnata. Non sono tuttavia presenti test automatici, pipeline CI/CD, gestione versioni delle dipendenze o configurazione ambienti.

16. Tracciabilità e fonti

Documento redatto sulla base dell'analisi dei file index.html (editor) e caa.html (pagina descrittiva) forniti. La pagina caa.html presenta il prodotto e ne illustra il flusso concettuale; le specifiche implementative sono state ricavate dal codice di index.html. Le affermazioni descrittive non confermate dal codice sono state trattate come obiettivi o presentazione, non come funzionalità verificate.

Riferimenti nel progetto: libreria it-compromise; API e risorse pittogrammi ARASAAC; Web Speech API del browser; CSS @media print.